

Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

Généré le	2026-09-24T09:39:56.9317654Z
Commande	go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1
Plateforme	windows/amd64
Processeur	AMD Ryzen 7 7735HS with Radeon Graphics
Profil CPU	cpu-20260924-093956967.prof
Profil mémoire	memory-20260924-093956967.prof

1. Résultats du benchmark

Benchmark	Moyenne	ns/op	B/op	allocs/op
BenchmarkTick-16	7.37 ms	7366943.8	54838.8	17

Détail hyperfine

Métrique	Valeur
Moyenne	4.9018 s
Écart-type	0.1102 s
Médiane	4.8987 s
Temps utilisateur	21.2681 s
Temps système	1.1909 s
Min	4.7607 s
Max	5.0326 s
Runs mesurés	5
Coefficient de variation	2.25%
Mémoire Hyperfine	Indisponible sur cette exécution

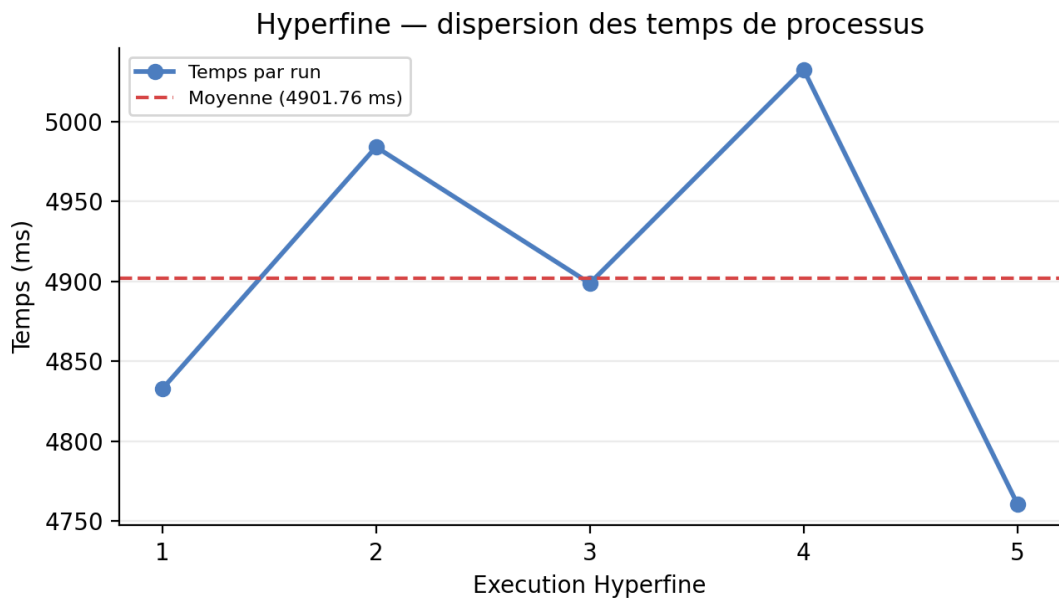


Figure — Temps de chaque exécution Hyperfine et moyenne.

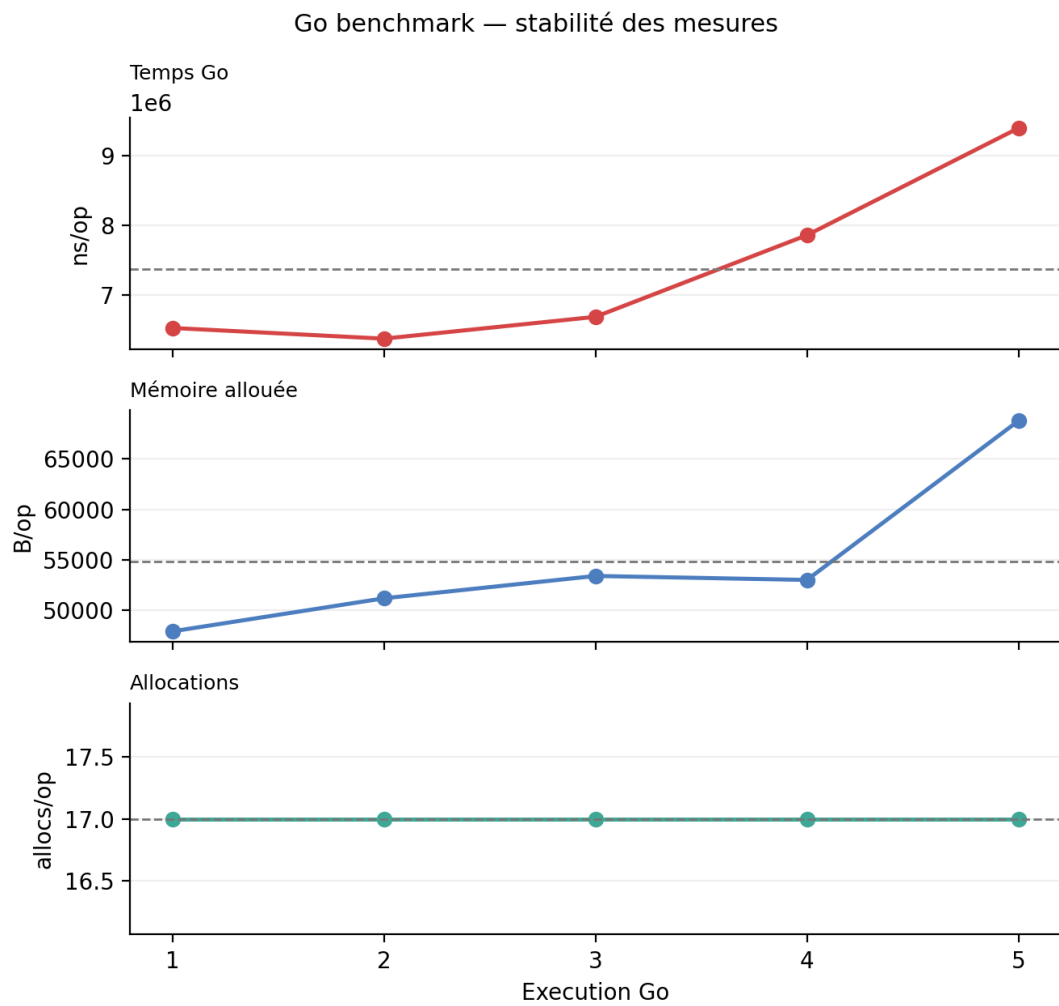


Figure — Dispersion des mesures Go : temps, mémoire allouée et allocations.

Résumé CPU et mémoire

Mesure	Valeur
CPU échantillonné (pprof)	32.22s
Mémoire allouée (pprof)	98.26MB
Heap vivant (pprof)	5128.09kB
Mémoire par opération	54838.8 B/op

Garbage collector Go

Cycles GC	70
Temps GC cumulé	69.730 ms
Plus longue pause	4.000 ms
Temps CPU GC cumulé	370.250 ms
Objectif mémoire maximal	95.00 MB

La mémoire pprof mesure le heap Go échantillonné (alloué ou encore vivant), pas toute la mémoire RSS du processus. Hyperfine peut fournir le RSS quand le backend système le supporte ; une valeur nulle est donc signalée comme indisponible.

2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **32.22s**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).infectionCandidate	26.55s	82.40%	29.91s	92.83%
jeu-de-la-vie/game.(*Board).index (inline)	2.75s	8.54%	2.76s	8.57%
jeu-de-la-vie/game.(*Board).Step	0.90s	2.79%	1.71s	5.31%
jeu-de-la-vie/game.(*Board).countInfectionCand	0.43s	1.33%	30.35s	94.20%
math/rand.(*Rand).Float64 (inline)	0.43s	1.33%	0.78s	2.42%
jeu-de-la-vie/game.squaredDistance (inline)	0.34s	1.06%	0.35s	1.09%
runtime.asyncPreempt	0.27s	0.84%	0.27s	0.84%
math/rand.(*rngSource).Uint64 (inline)	0.19s	0.59%	0.19s	0.59%

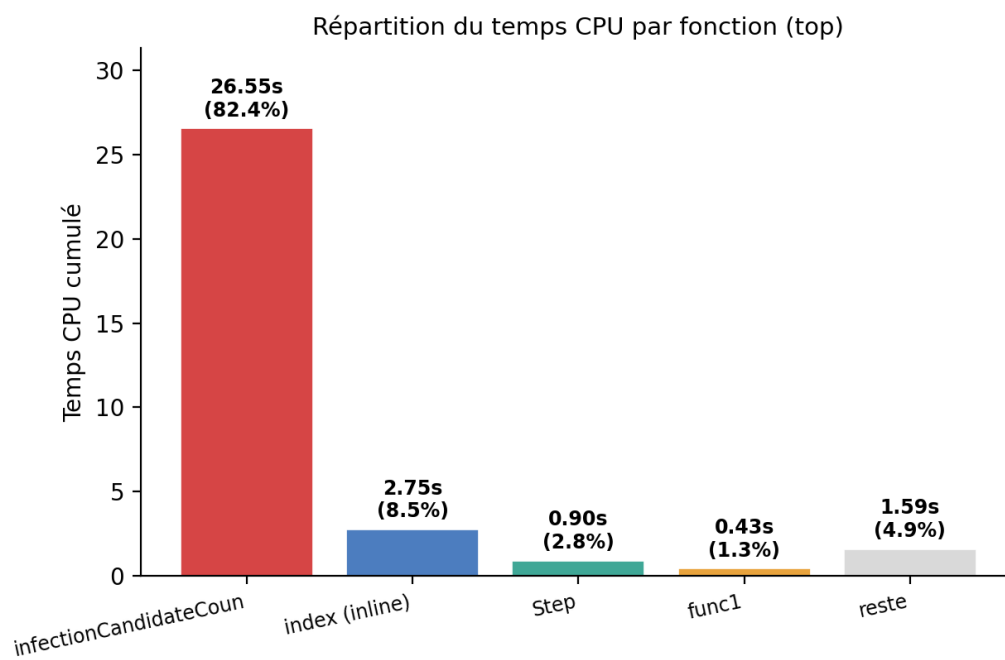


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

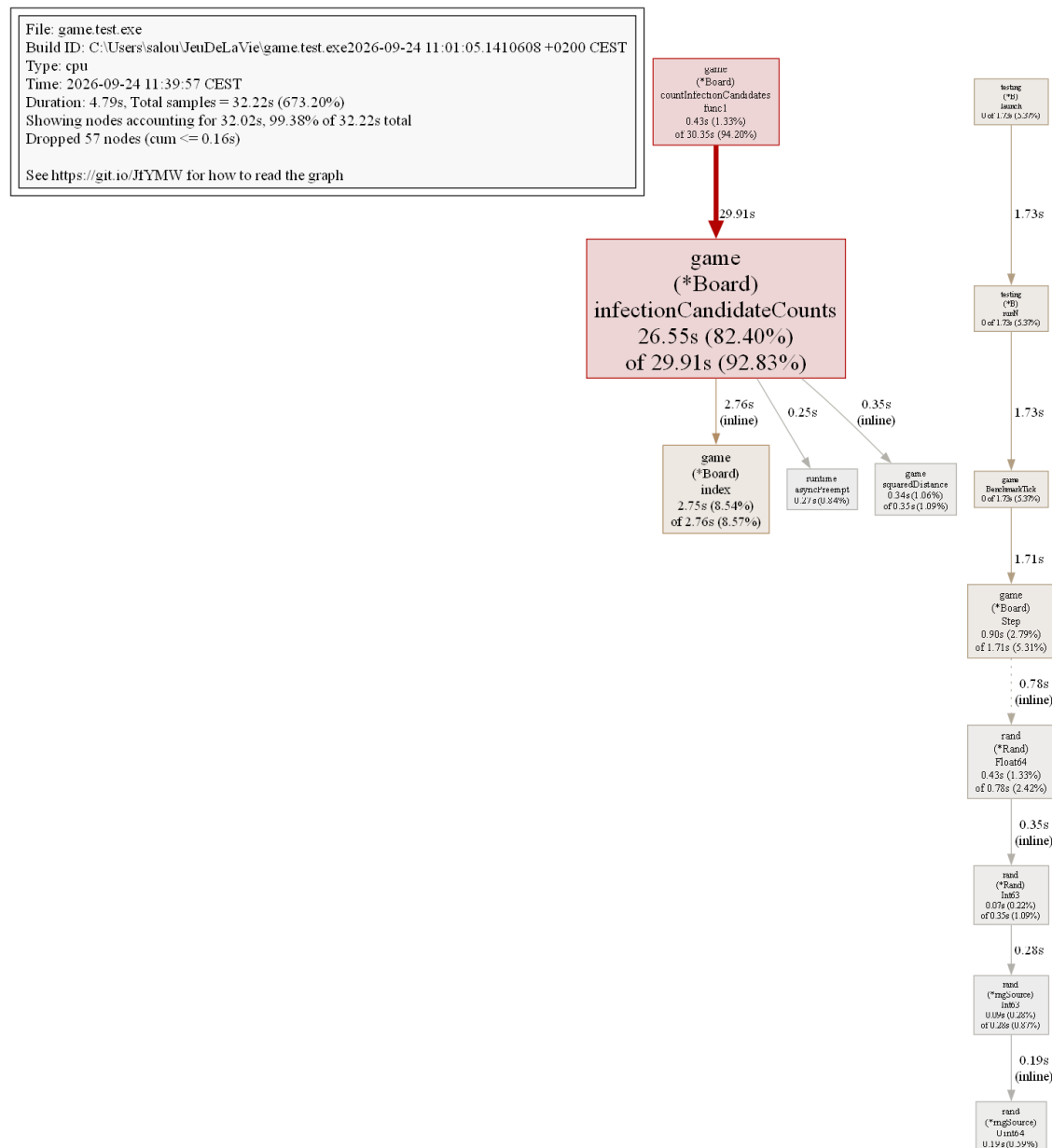


Figure — Graphe d'appels CPU (go tool pprof -png).

2.1 Diagnostic automatique

La fonction dominante de ce run est **jeu-de-la-vie/game.(*Board).infectionCandidateCounts**, responsable de **82.4%** du temps CPU total (flat), pour **92.8%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en $O(n)$ ou $O(\log n)$ équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

Rappel — pourquoi la complexité algorithmique domine
à mesure que les données grandissent (schéma, non mesuré)

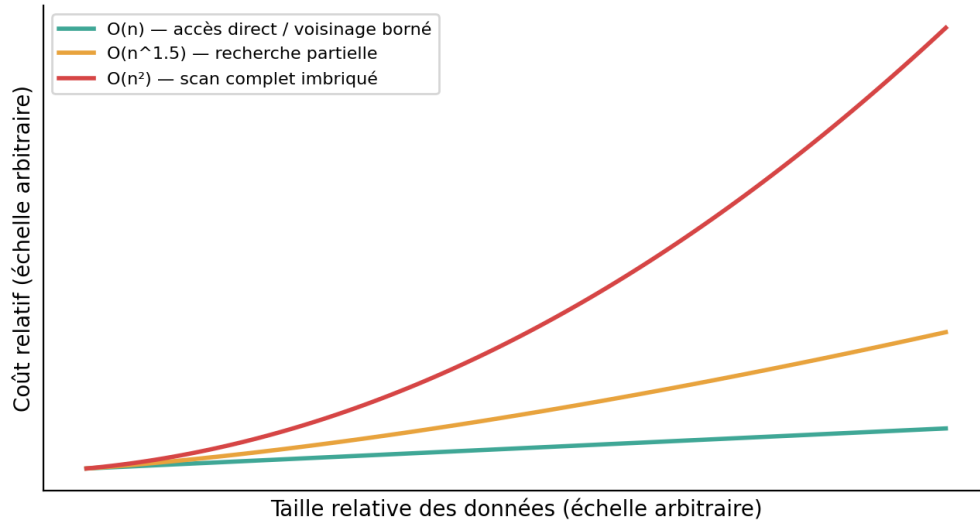


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **98.26MB**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).ensureScratch (inl	51.78MB	52.69%	51.78MB	52.69%
jeu-de-la-vie/game.NewBoard (inline)	19.33MB	19.68%	19.33MB	19.68%
jeu-de-la-vie/game.RandomBoard	16.57MB	16.86%	35.90MB	36.54%
runtime.mallocgc	4.51MB	4.59%	4.51MB	4.59%
runtime/pprof.StartCPUProfile	1.16MB	1.18%	1.16MB	1.18%
jeu-de-la-vie/game.(*Board).countInfectionCand	1MB	1.02%	1MB	1.02%
compress/flate.NewWriter (inline)	0.88MB	0.90%	1.41MB	1.44%
compress/flate.newDeflateFast (inline)	0.53MB	0.54%	0.53MB	0.54%

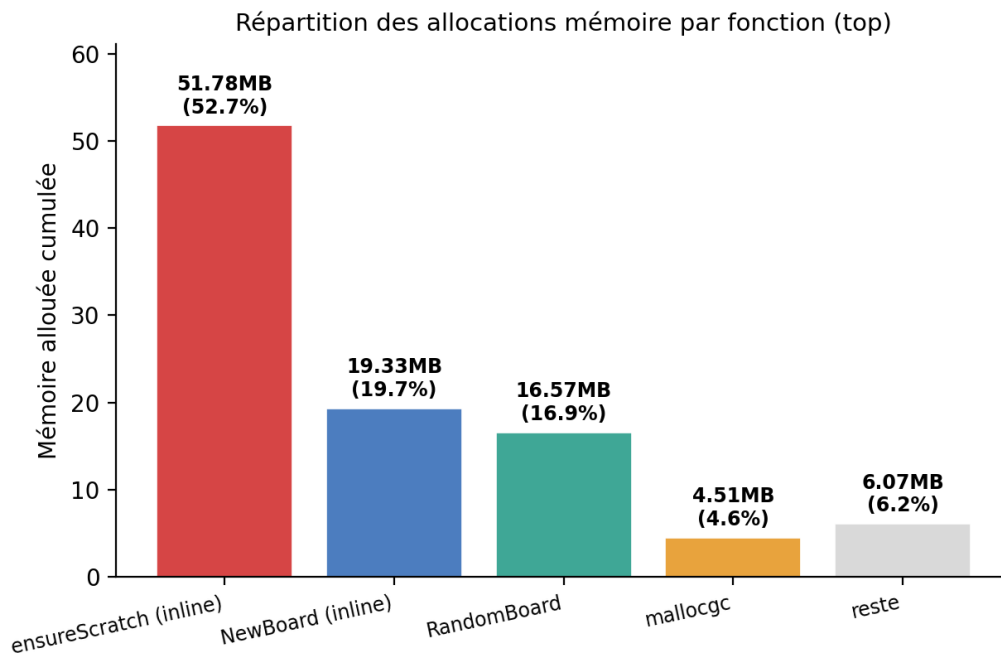


Figure — Répartition des allocations mémoire par fonction.

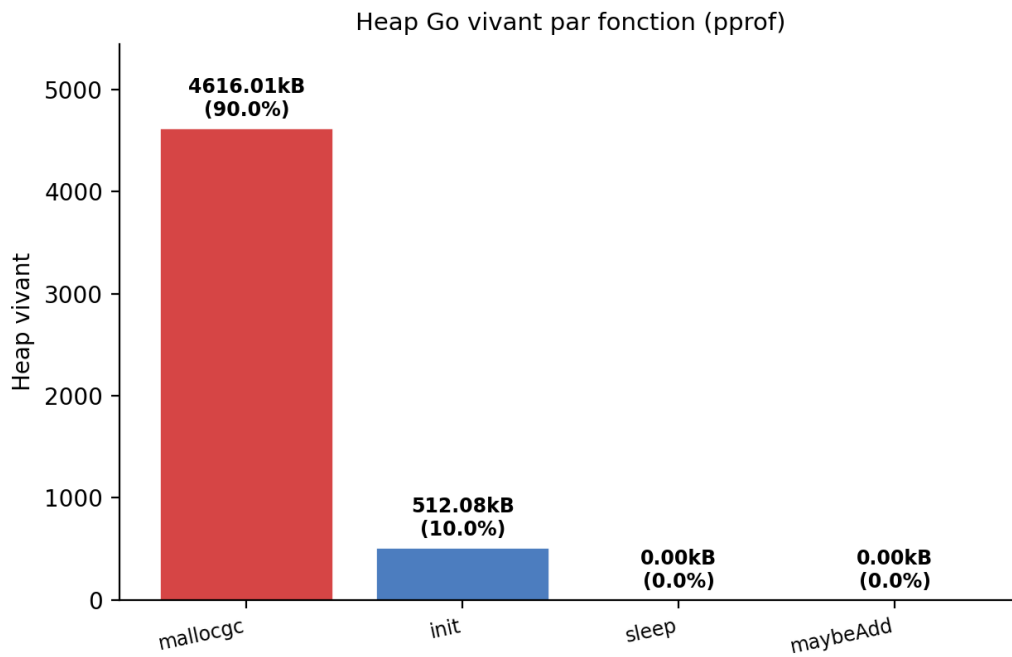


Figure — Heap Go encore vivant par fonction (pprof -inuse_space).

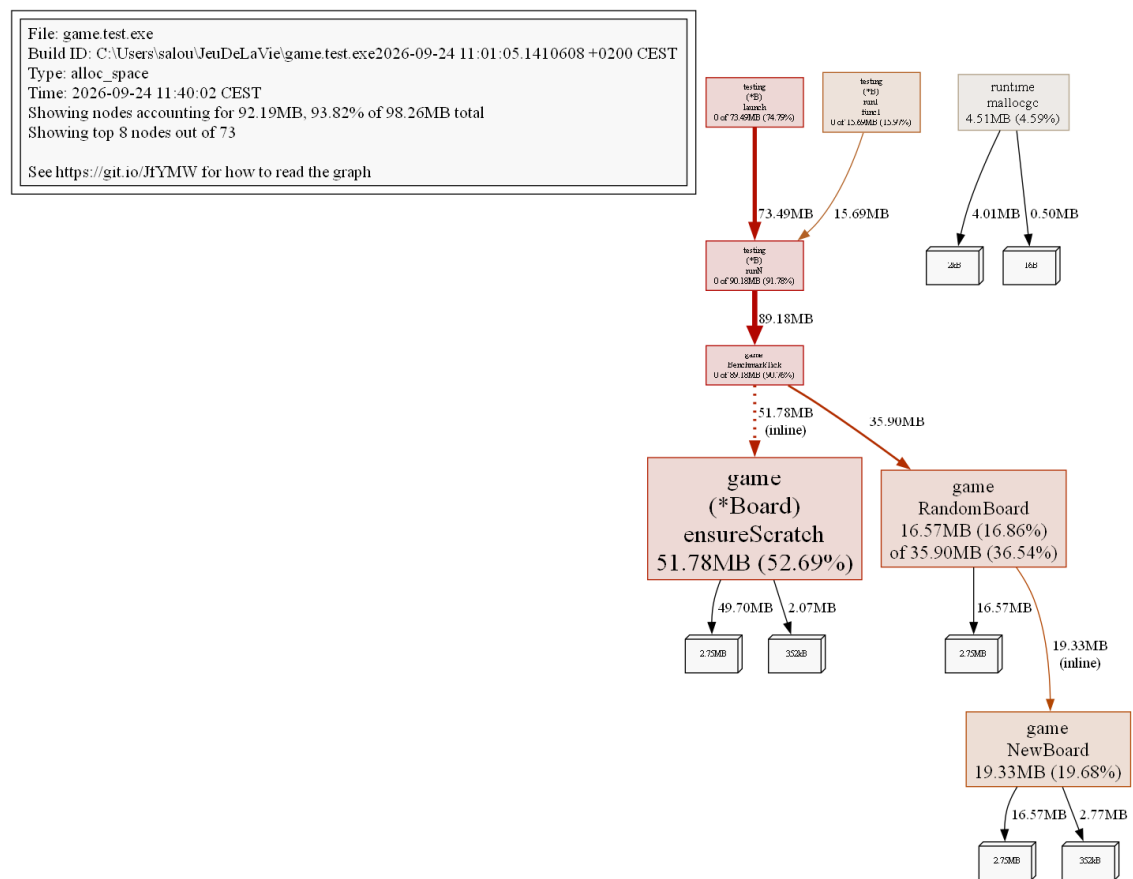


Figure — Graphe d'appels mémoire (go tool pprof -png).

4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.

Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte 600×600 .
- La carte et le générateur aléatoire utilisent toujours la seed 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- La trace GC détaillée est enregistrée dans le fichier `gc-\*.log` et résumée dans le PDF.
- Les métriques I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

Rapport généré automatiquement le 2026-09-24 11:40:44 par `generate_benchmark_report.py`, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec `go tool pprof`.